



BITS Pilani
Pilani Campus

API-driven Cloud Native Solutions

Dr. Shreyas Rao
Associate Prof. (Off Campus), CSIS, BITS-Pilani



CC ZG506

Lecture No. 12

Contact Session wise Coverage

Session	Coverage	
CS01	API Basics (OpenAPI, REST, gRPC, GraphQL)	}
CS02	API Basics + Cloud Native Application	
CS03	Cloud Native Application	
CS04	Introduction to Data Science & Machine Learning, DataOps & MLOps	}
CS05	API driven Cloud-native Data Pipeline	
CS06	API-driven ML pipelines	
CS07	Tools review	}
CS08	Basic APIs for AI Applications - NLP, CV, Speech recognition	
CS09	Language Models (LLMs)	
CS10	Fine Tuning + RAG Models	
CS11	RAG Models	
CS12	Cloud Native Application Deployment	}
CS13	IoT and Data Analytics	
CS14	APIs for IoT and Data Analytics	
CS15	Serverless Functions; Case study	
CS16	Course review and discussion	

Agenda

Part-C: Cloud Native Application Deployment

- Containers
- Dockers
- Kubernetes
- Deployment Strategies
- Demos

Singer Analogy

01

A Singer is invited for a concert. He goes there and Mike does not work

02

Singer is invited to second concert, where Speakers do not work

03

Singer now invests in his own mike and speakers, and takes them to whichever concert he goes.

This way the vagaries of the environment (mike, speaker) are avoided

What are Containers

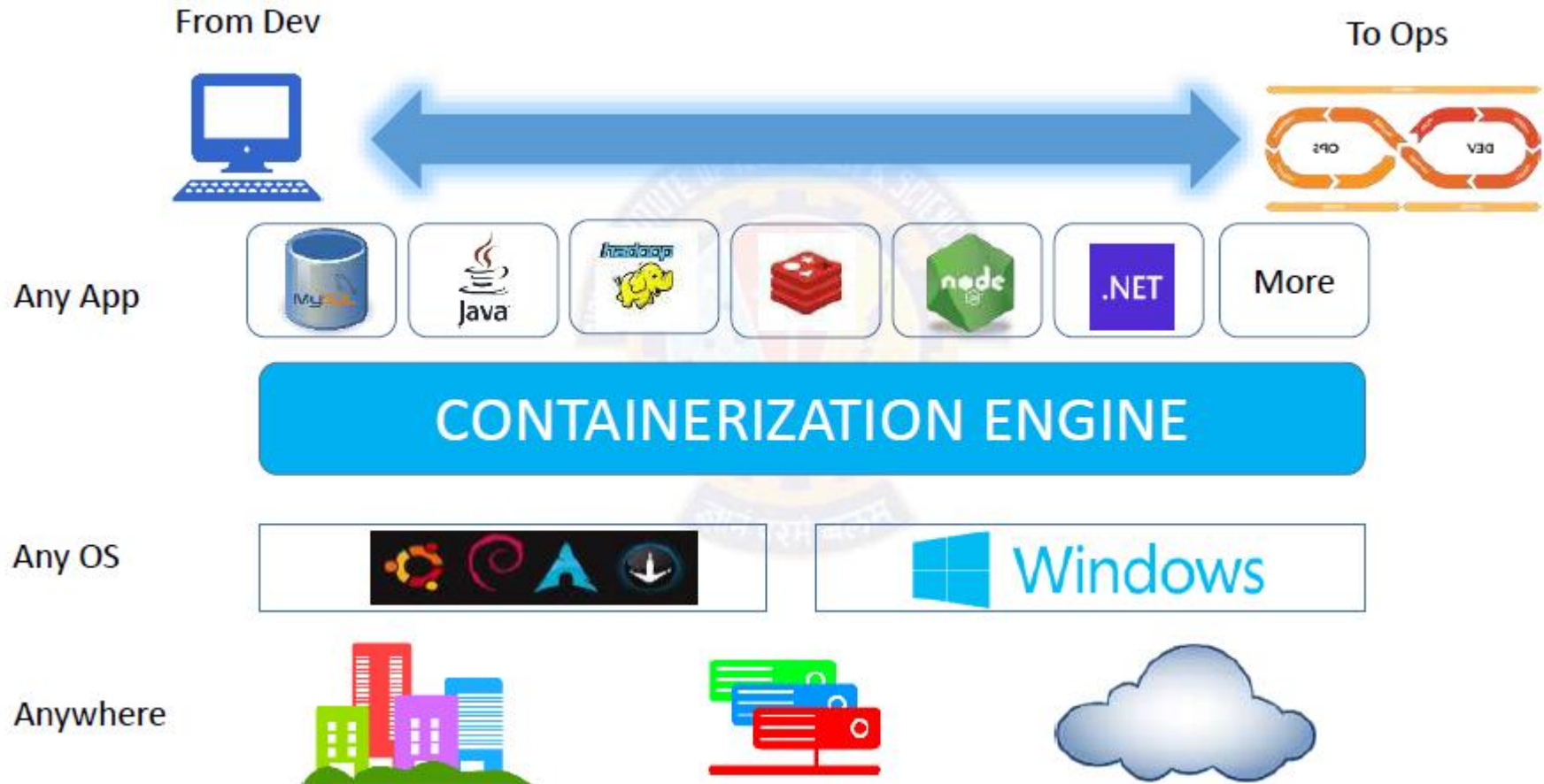
- A software container is a standardized package of software
- Everything needed for the software to run is inside the container
- The software code, runtime, system libraries, and settings are all inside a single container
- **Container based deployments is favoured for Microservices**

Cloud Native Development

1. Microservices Architectural Style
- 2. Containers for Deployment**
3. Cloud
4. DevOps

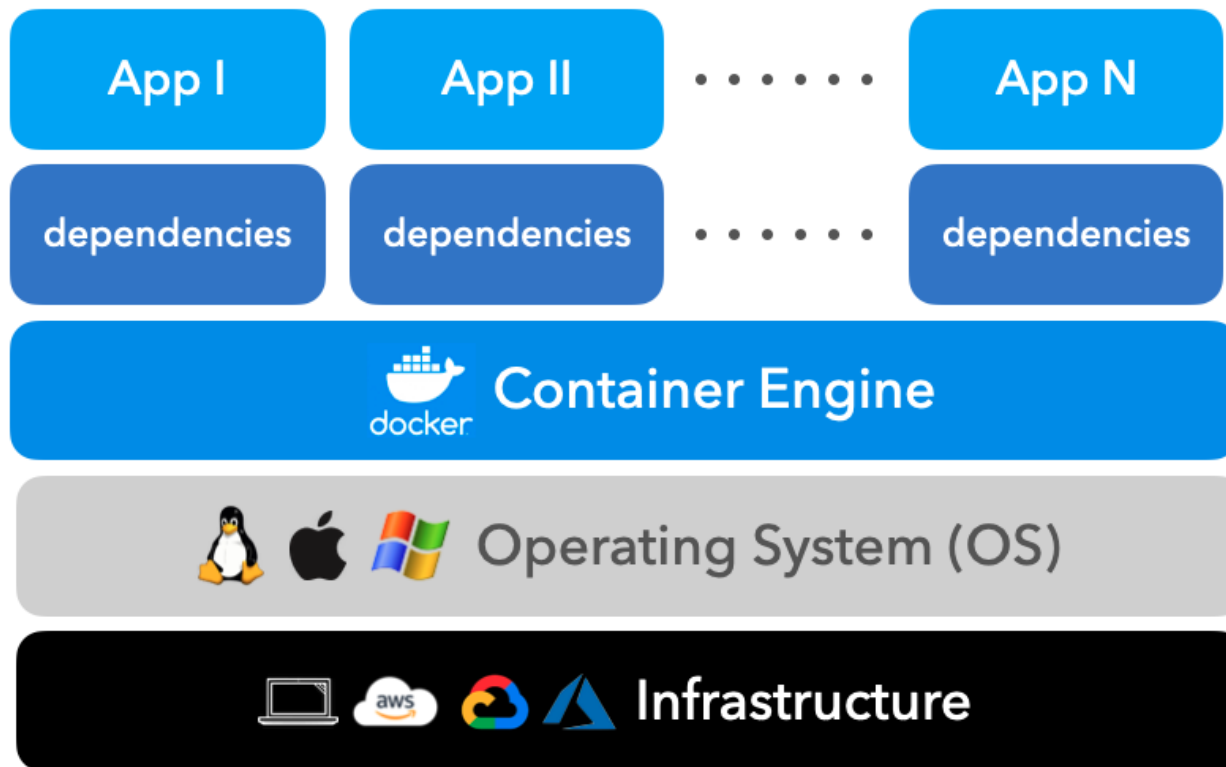


Containers – Build, Ship, Run Any App Anywhere

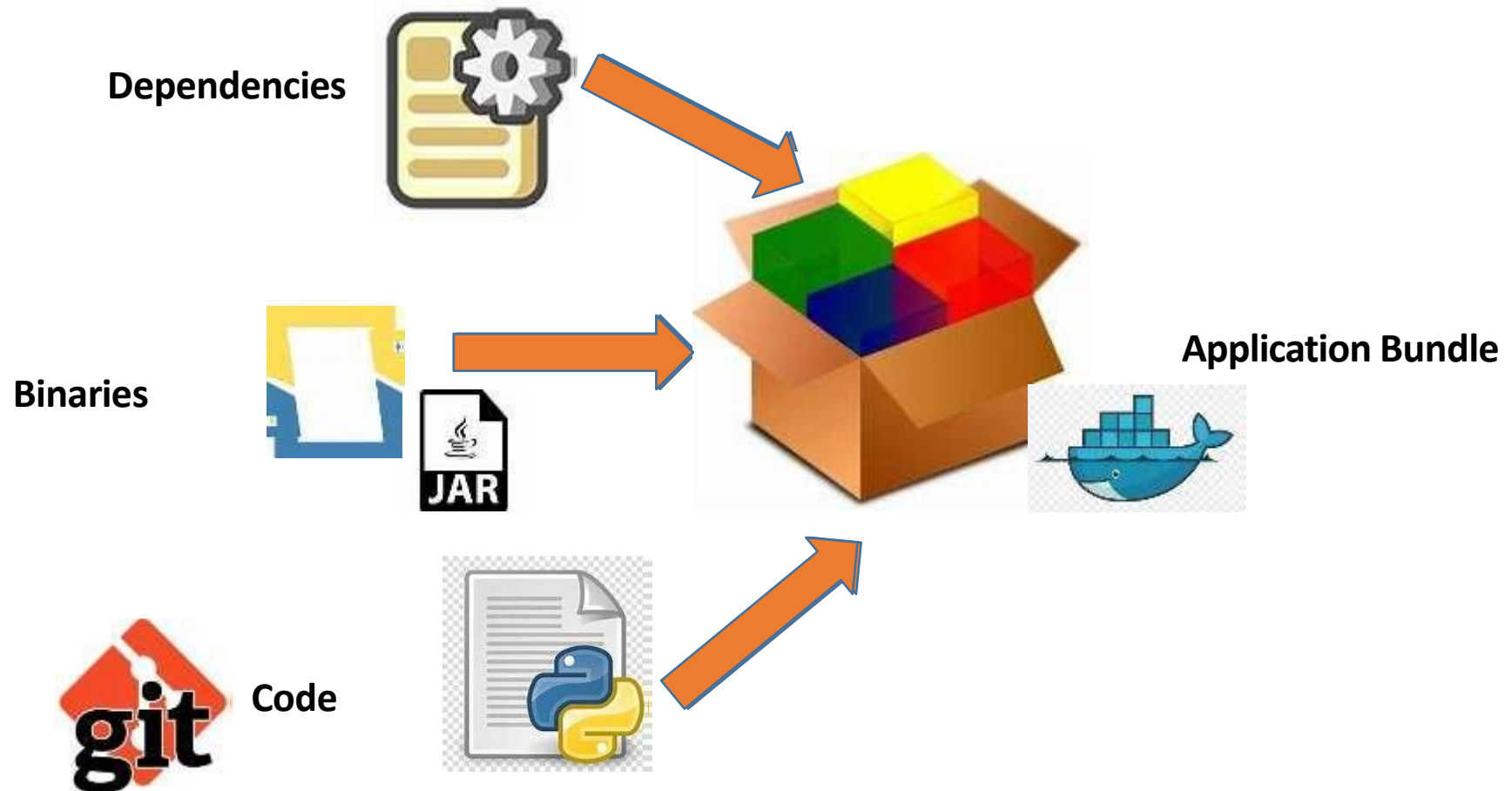


Docker

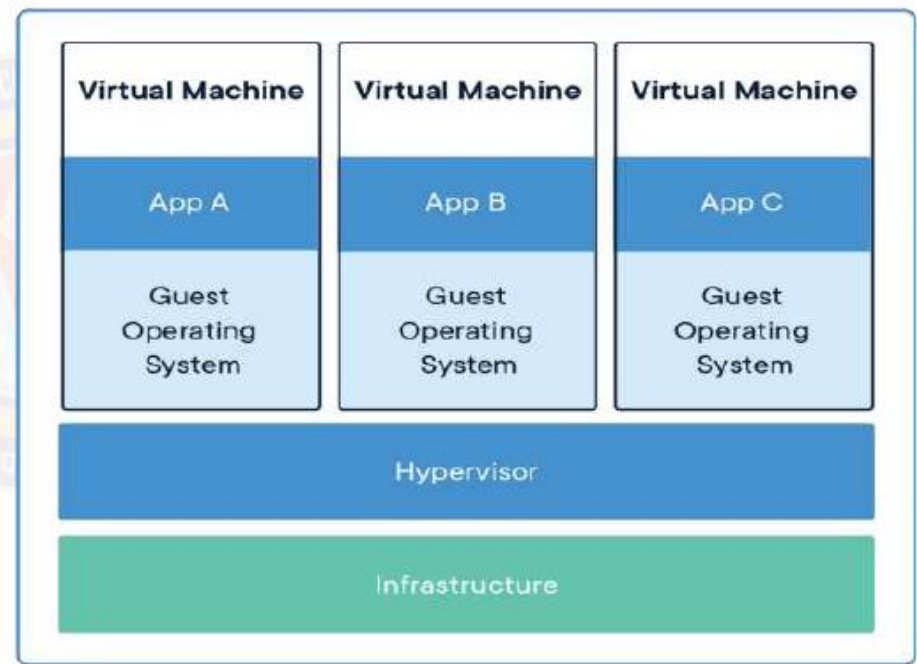
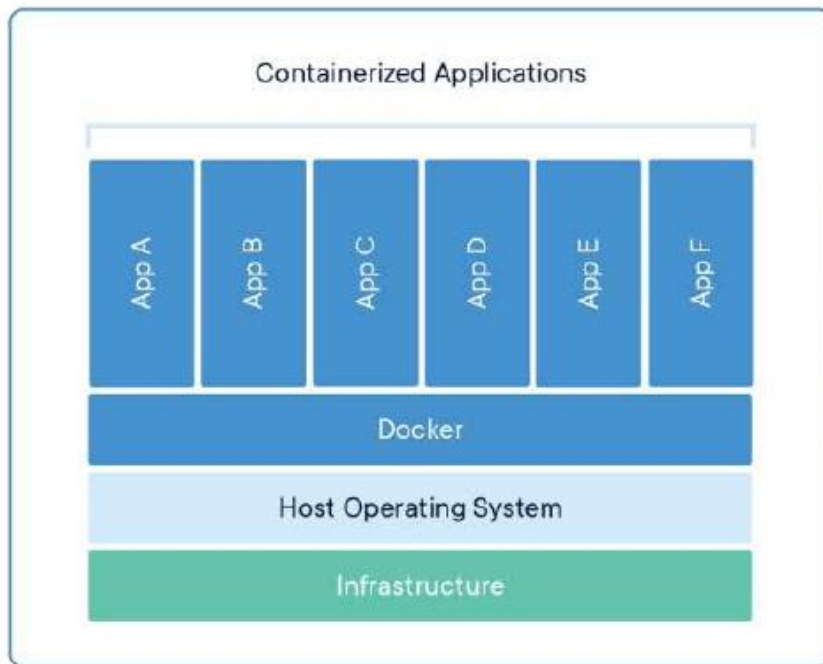
- Is a popular Container technology
- It is lightweight, portable and self-sufficient container



Docker - Container to Package Software



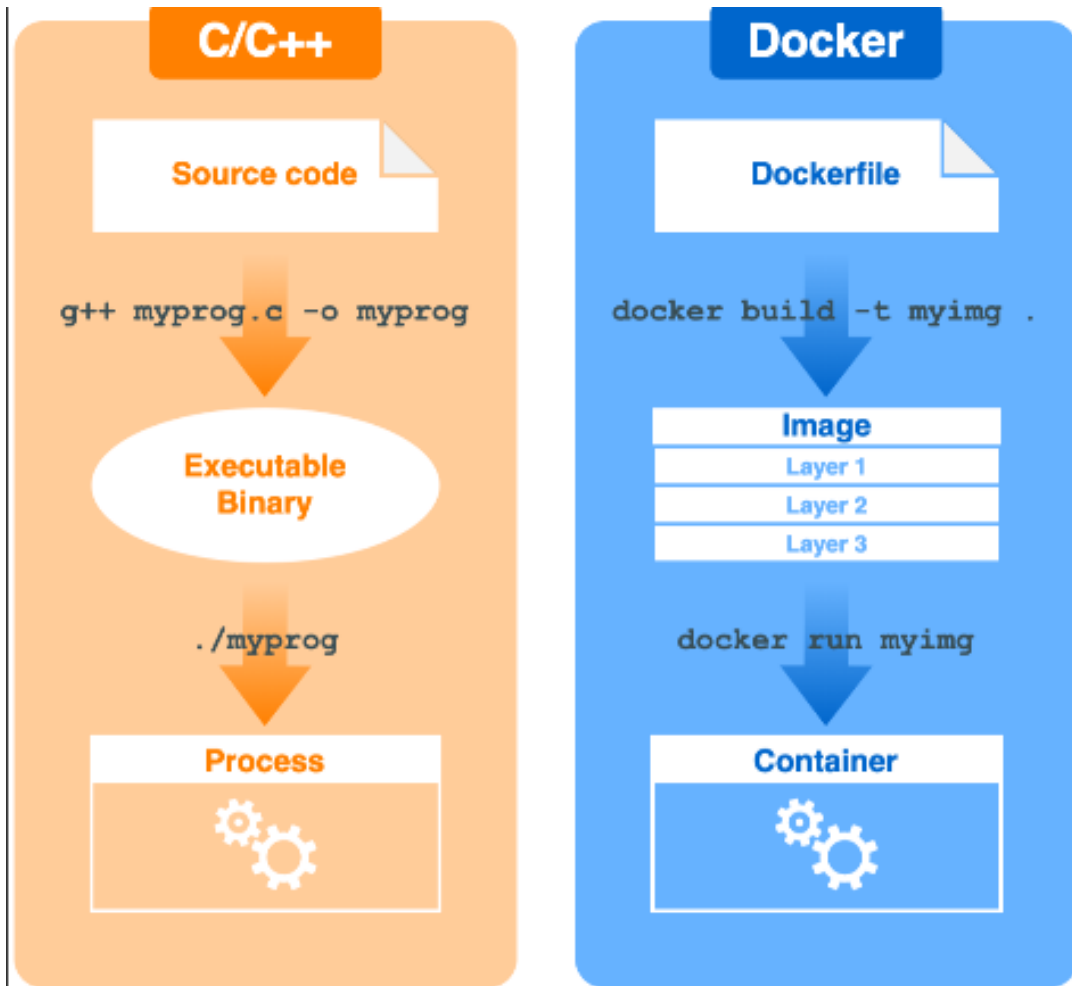
Containers vs Virtual Machines



Basic Definitions

- Docker - It is a way to package Software so that it runs on any hardware (OS & hardware).
- Requires knowledge of three things - *dockerfile, images and containers*.
 - a. Docker File - Blueprint for building a Docker Image. **Set of Instructions**
 - b. Docker Image - Template for running Docker Containers. Docker image is a lightweight, standalone, and executable package that includes everything needed to run a piece of software, including the code, runtime, libraries, and system tools.
 - c. Docker Container - A docker container is a portable unit of software - that has the application, along with the associated dependency and configuration. It is running process of the image. May include Application (Web - Node.js) and Database (DB - MySQL). One image file can be made to spawn multiple processes across environments.

Docker



Step 1: Dockerfile

**Step 2: Build Image
[docker build]**

**Step 3: Run Image as Container
[docker run]**

Purpose of Docker

- Reproduce the environment in Development, Staging and Production.
- We can define the environment with the Docker File.
- People can use the dockerfile to create an image. Images can be uploaded to Cloud (public or private) repositories using “docker push” command and pulled using “docker pull” command from the repository.

Docker Architecture

- Docker uses a client-server architecture.
- The Docker daemon
 - The Docker daemon (dockerd) listens for Docker API requests
 - Manages Docker objects such as images, containers, networks, and volumes.
 - Builds, runs, and distributes containers
- The Docker client
 - The Docker client talks to the Docker daemon
 - The Docker client and daemon can run on the same system
 - The Docker client can communicate with more than one daemon..
- The Docker client and daemon communicate using a REST API, over UNIX sockets or a network interface.

The diagram illustrates the Docker architecture components and their interactions:

- Client:** Contains three main components: `docker build`, `docker pull`, and `docker run`.
- DOCKER_HOST:** The central hub containing the `docker daemon`, `Containers`, and `Images`.
 - `Containers` are represented by three blue boxes with the Docker logo.
 - `Images` are represented by two icons: the Docker logo and the OpenStack logo.
- Registry:** Contains the `docker daemon`, the Docker logo, the NGINX logo, and the OpenStack logo.

Interactions:

- build:** Indicated by a dotted line from `docker build` to the `Images` section in `DOCKER_HOST`.
- pull:** Indicated by a dashed line from `docker pull` to the `Images` section in `DOCKER_HOST`.
- run:** Indicated by a solid line from `docker run` to the `Containers` section in `DOCKER_HOST`.
- There are also dashed lines from the `Images` section in `DOCKER_HOST` to the `Registry`, and from the `Registry` to the `Images` section in `DOCKER_HOST`.

Legend:

- build
- pull
- run

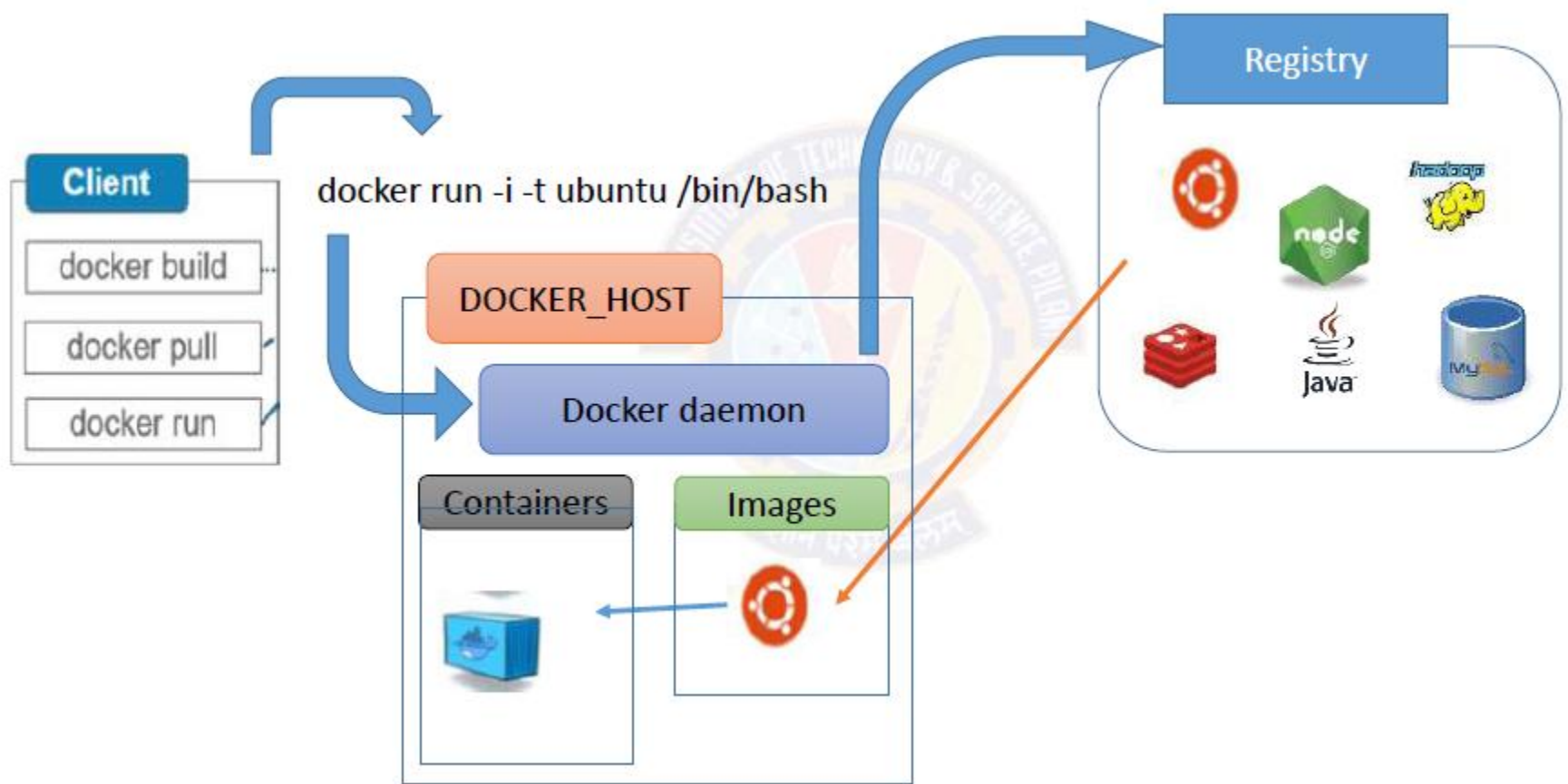
Docker Desktop

- Docker Desktop is a one-click-install application for Mac, Linux, or Windows environment that enables you to build and share containerized applications and microservices.
- It provides a straightforward GUI (Graphical User Interface) that lets you manage your containers, applications, and images directly from your machine.

Docker Hub

- Docker Hub is the world's largest library and community for **container images**
- Contains Official Images for MongoDB, NodeJS, Redis, Ubuntu, Python, MySQL etc.
- URL - <https://hub.docker.com/>

Example for running a Docker Container



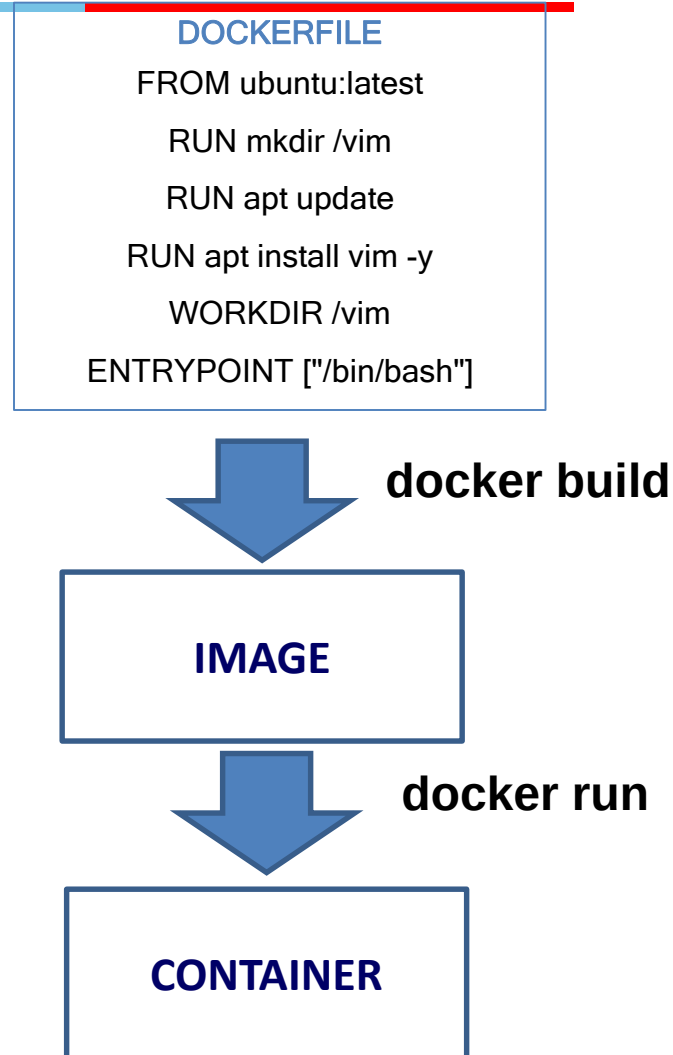
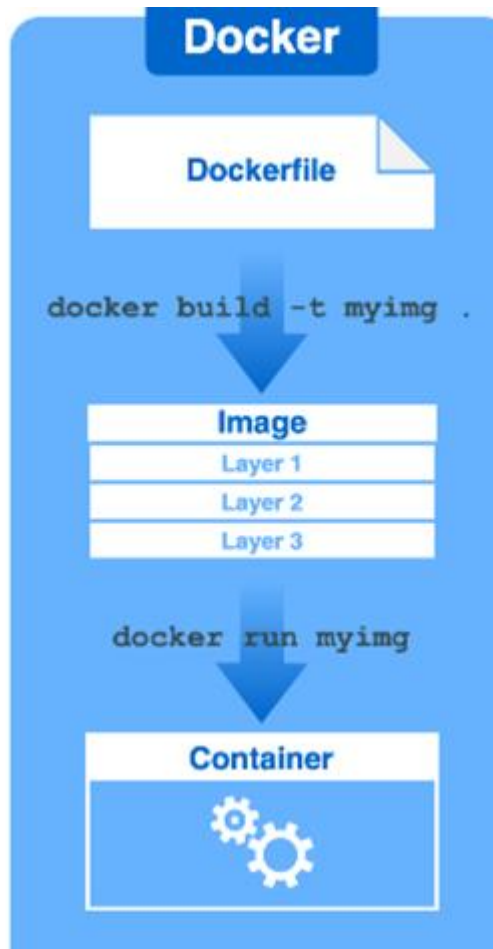
Dockerfile

Dockerfile is a text file, named *Dockerfile*, that includes specific keywords that dictate how to build a specific image.

Keywords are:

- **ADD** copies the files from a source on the host into the container's own filesystem at the set destination.
- **CMD** can be used for executing a specific command within the container.
- **ENTRYPOINT** sets a default application to be used every time a container is created with the image.
- **ENV** sets environment variables.
- **EXPOSE** associates a specific port to enable networking between the container and the outside world.
- **FROM** defines the base image used to start the build process.
- **MAINTAINER** defines a full name and email address of the image creator.
- **RUN** is the central executing directive for Dockerfiles.
- **USER** sets the UID (or username) which is to run the container.
- **VOLUME** is used to enable access from the container to a directory on the host machine.
- **WORKDIR** sets the path where the command, defined with **CMD**, is to be executed.
- **LABEL** allows you to add a label to your docker image.

Docker - Example



Docker Demo

File: “Docker Basics - Docker Desktop and DockerHub”



KUBERNETES



Kubernetes / K8s

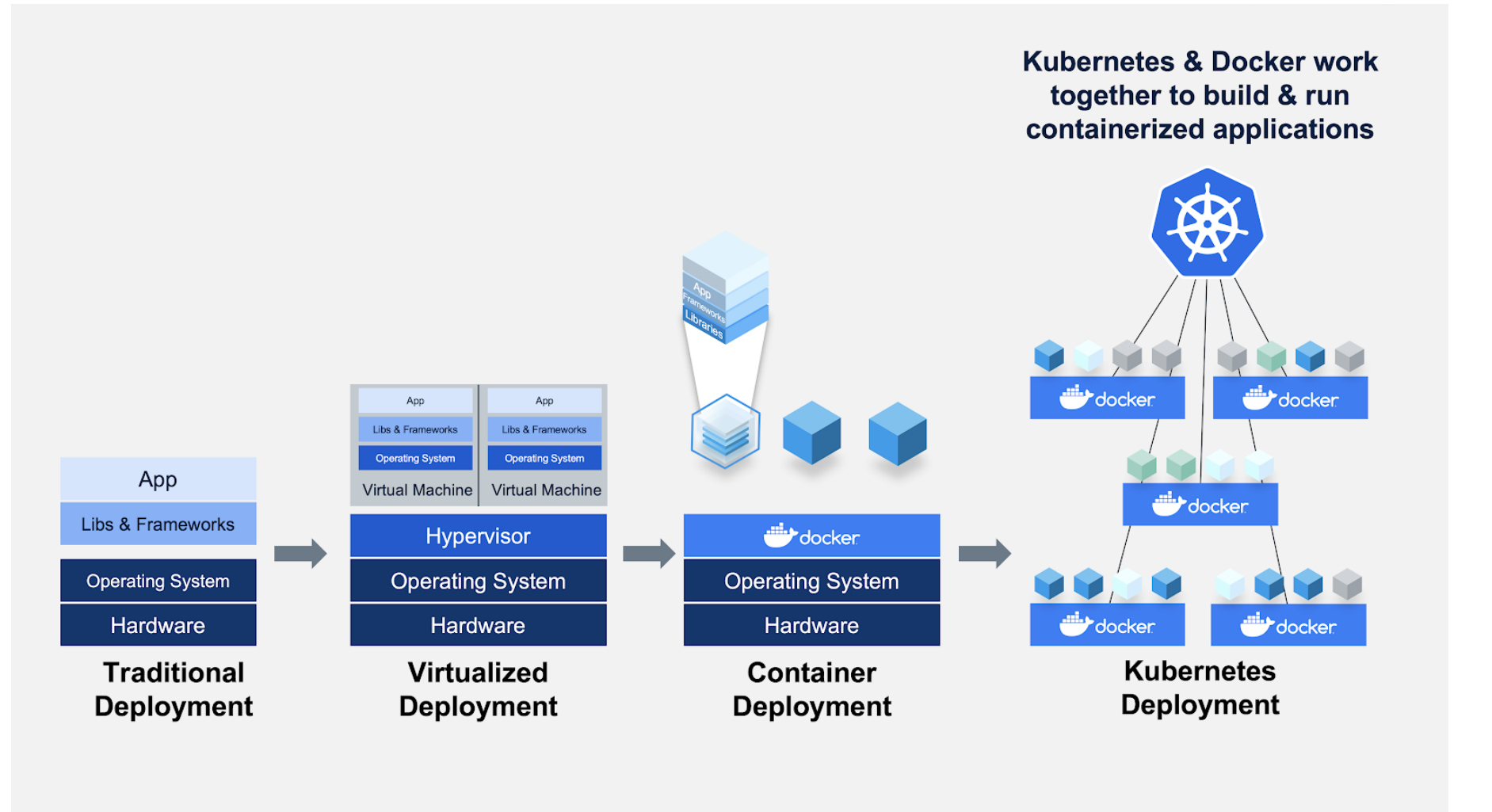
- Kubernetes is a *container orchestration tool* that manages applications/ services available on a container platform like Docker
- Developed by Google labs and later donated to CNCF (Cloud Native Computing Foundation).
- First CNCF project to graduate
- Open source
- Written in Golang



kubernetes



Evolution of Deployment



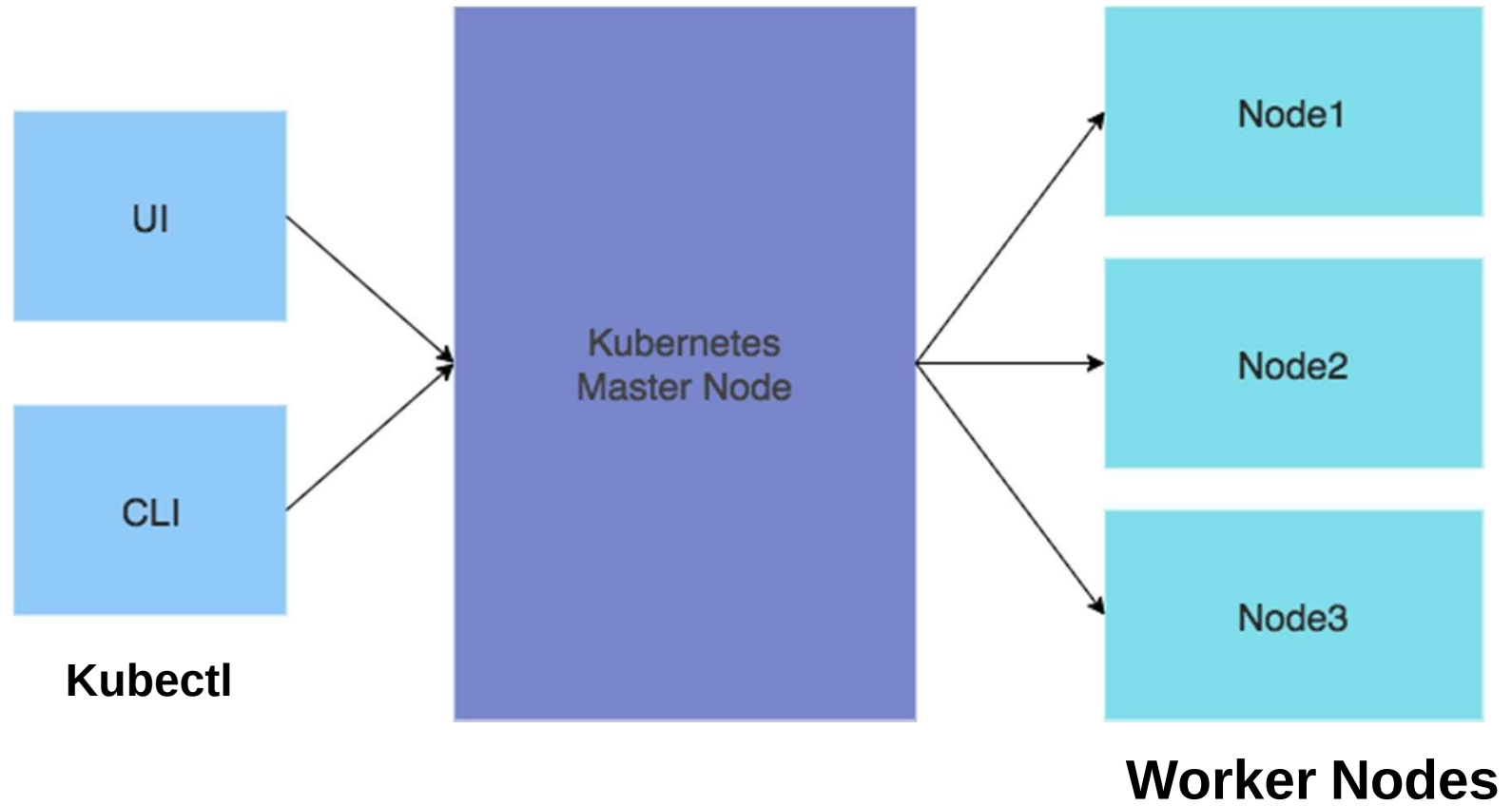
Features of Kubernetes (Container Orchestrators)

1. High Availability or No downtime
2. Scalability or High Performance
3. Disaster Recovery - Backup and Restore

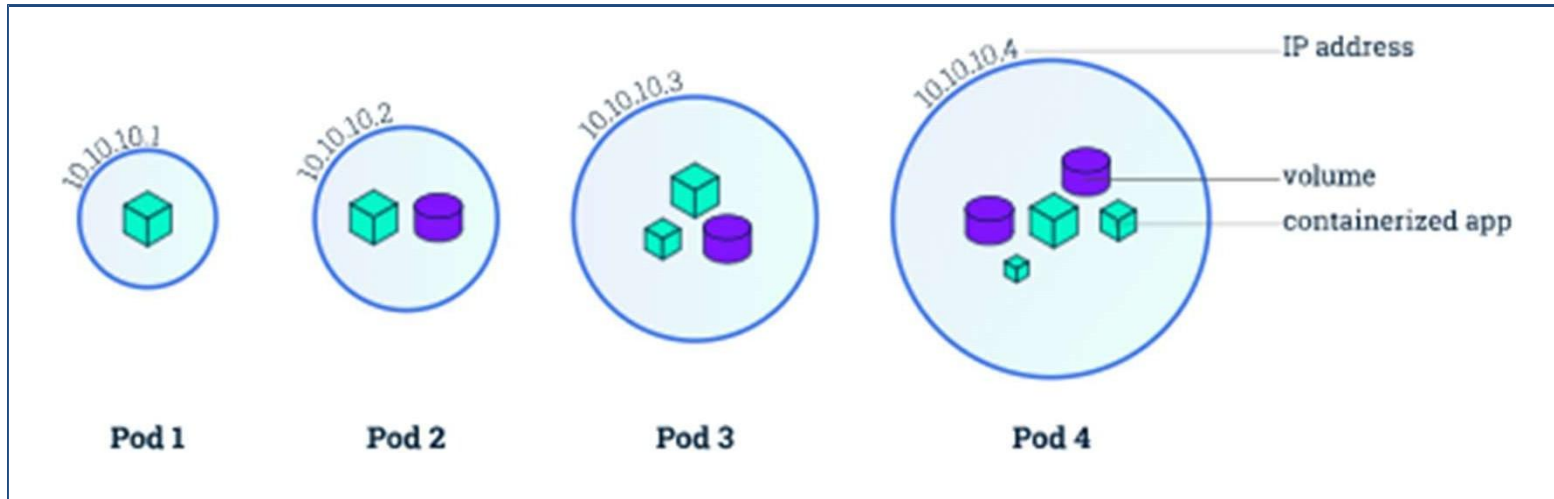


kubernetes

Kubernetes Components



Kubernetes Components

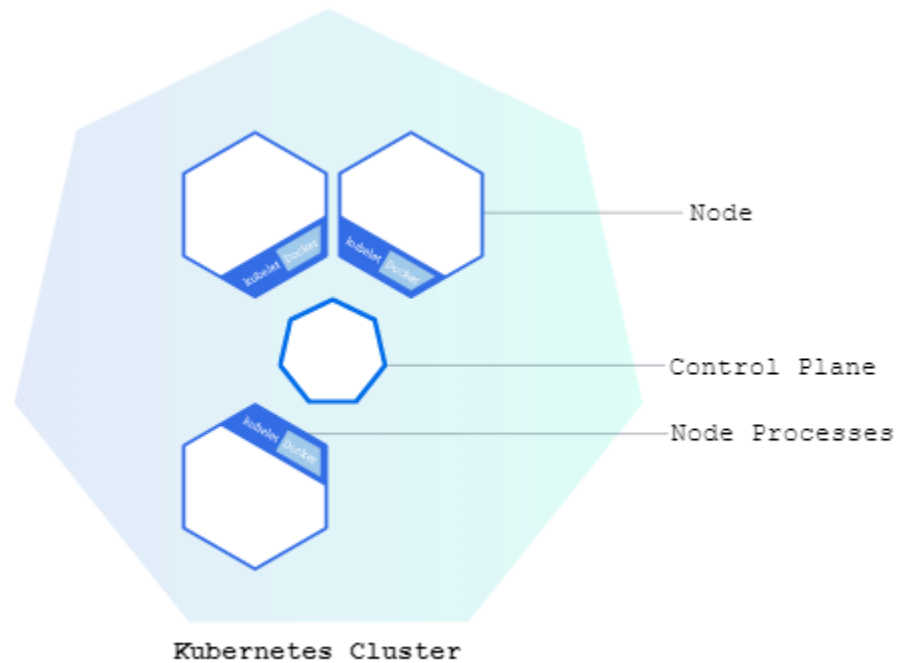


Node

Pods are the smallest deployable units of computing that you can create and manage in Kubernetes.

A *Pod* is a group of one or more containers, with shared storage and network resources

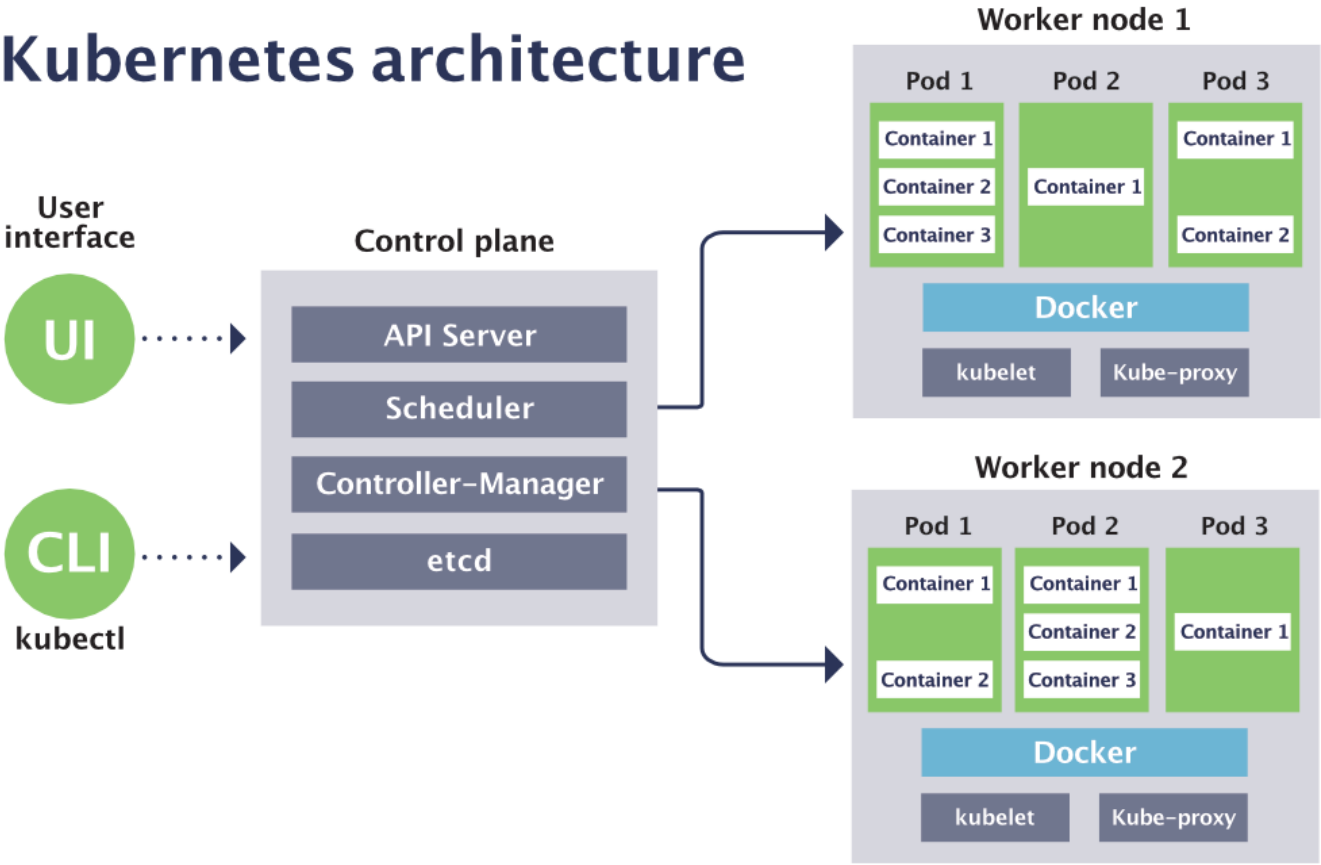
Kubernetes Cluster



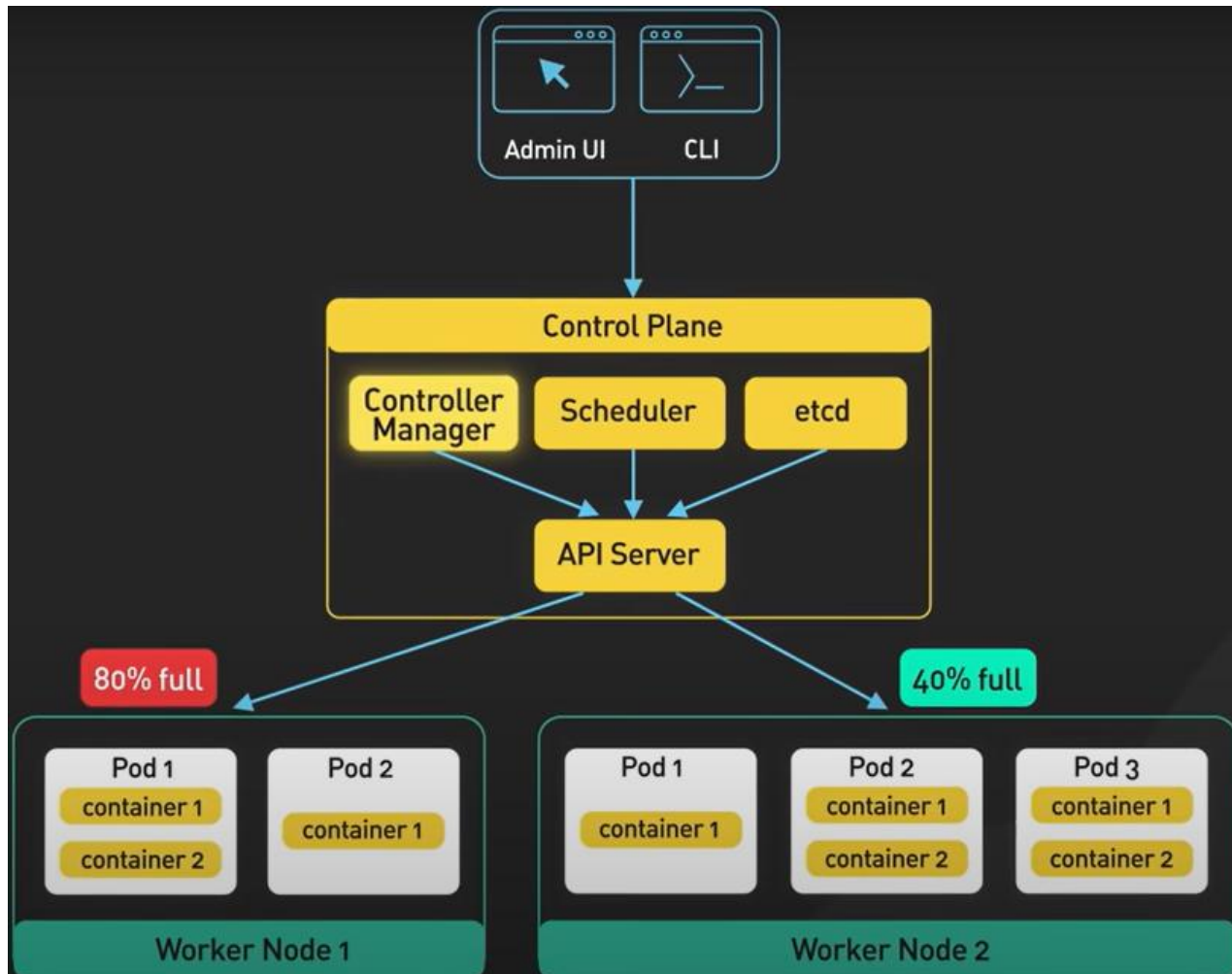
Control Plane nodes are also called as 'Master Nodes'

Kubernetes Architecture

Kubernetes architecture



Scheduler – Schedules Pods on Cluster nodes





Kubernetes Components

- **Master Node:** The master node (Control Plane Node) is responsible for managing the entire cluster. It contains various components like the API server, etcd, controller manager, and scheduler.
- **Worker Nodes:** These are the nodes where containers run. They are responsible for running the application workload and communicating with the master node.
- **Pods:** Pods are the smallest deployable units in Kubernetes. They are used to encapsulate one or more containers and share storage and network resources.
- **Services:** Services are used to expose a group of pods to the network. They provide a stable IP address and DNS name that other services and pods can use to communicate with the group of pods.
- **Controllers:** Controllers are used to manage the lifecycle of pods and services. They ensure that the desired number of replicas of a pod or service are running and will automatically create or delete pods as necessary.

Master Node Components

- **API Server:** The API server is the central control point for the Kubernetes cluster. It exposes the Kubernetes API, which is used by other components to communicate with the cluster. It stores the state of the cluster in etcd.
- **etcd:** etcd is a distributed key-value store that stores the configuration data of the Kubernetes cluster. It is the source of truth for the current state of the cluster and is used by the API server, controller manager, and scheduler.
- **Controller Manager:** The controller manager is responsible for running the various controllers that regulate the state of the cluster. For example, the ReplicaSet controller ensures that the desired number of pod replicas are running, and the Deployment controller manages rolling updates of deployments.
- **Scheduler:** The scheduler is responsible for assigning pods to nodes in the cluster. It takes into account factors such as resource requirements, node availability, and user-defined policies.



How Deployment works in Kubernetes

- **Kubectl** - The Kubernetes command-line tool, kubectl, allows you to run commands against Kubernetes clusters. You can use kubectl to deploy applications, inspect and manage cluster resources, and view logs.
- **Kubelet** - Manages the state of the node and ensures that containers are running and healthy. It also monitors the state of the containers and restarts them if they fail.
- **Kube-proxy** - Manages network connectivity to and from the Pods

<https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>

Minikube

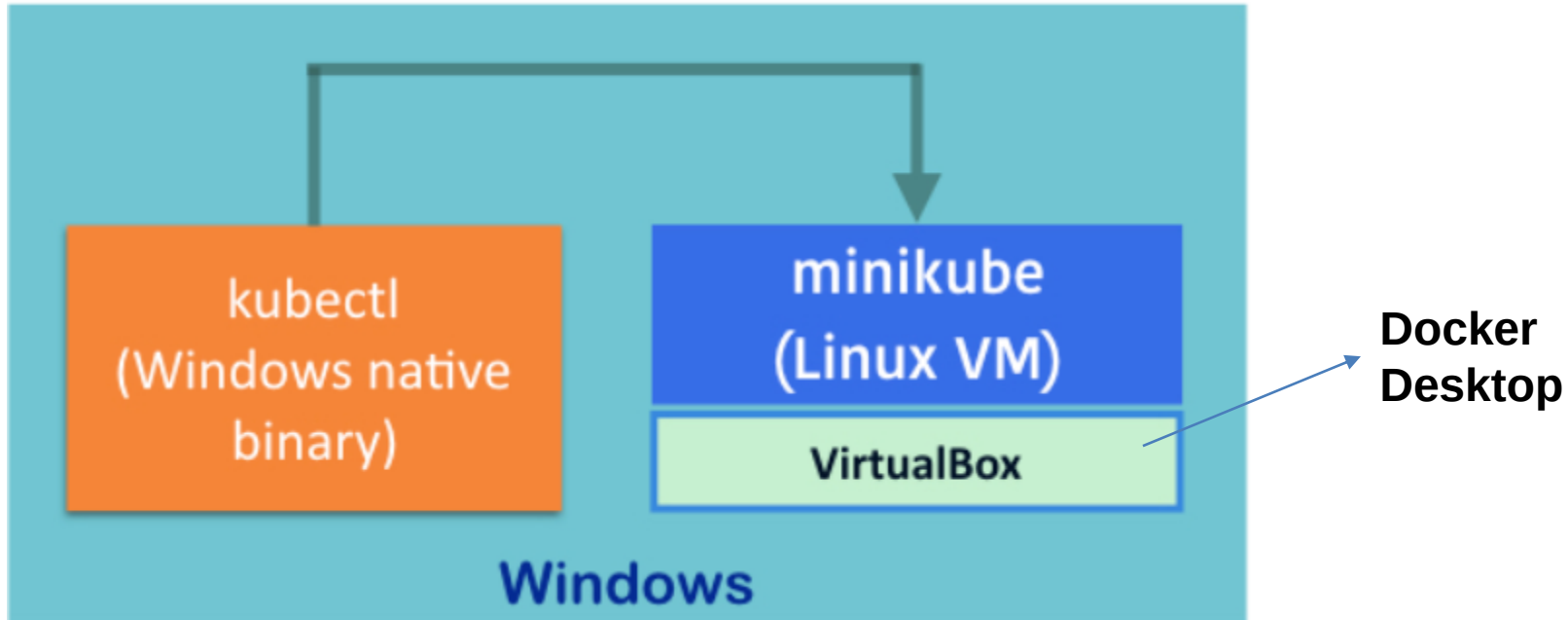
- Is a **Single Node Kubernetes cluster**, which we can create on local machine
- Helps in development and experimentation
- Can be run inside of a Container (Docker) or Virtual Machine



minikube

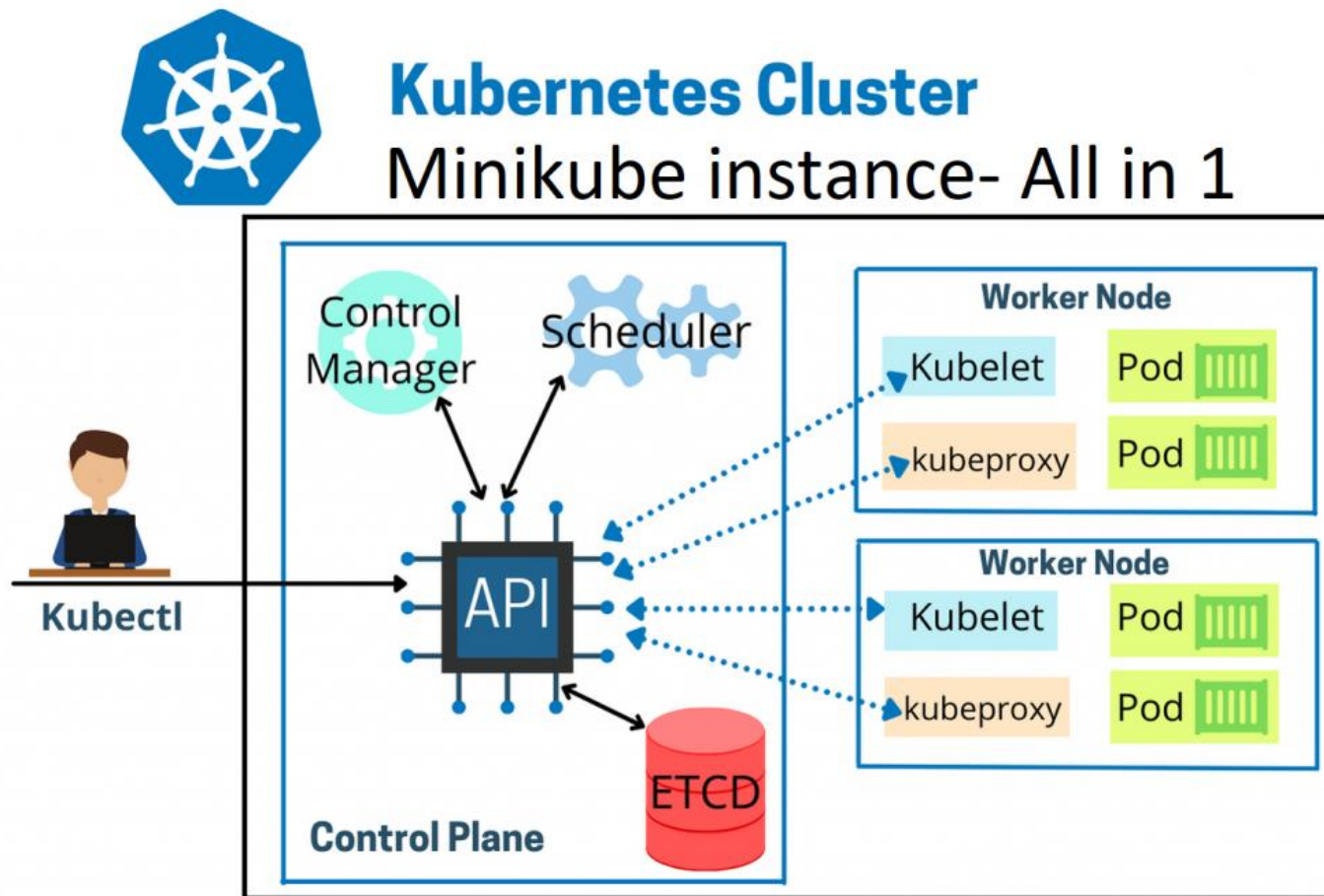
<https://minikube.sigs.k8s.io/docs/>

Minikube Architecture



Container or virtual machine manager, such as: Docker, QEMU, Hyperkit, Hyper-V, KVM, Parallels, Podman, VirtualBox, or VMware Fusion/Workstation

Minikube Architecture



Layers of Abstraction in Kubernetes

Layers of Abstraction



Deployment manages a ..



ReplicaSet manages a ..



Pod is an abstraction of ..



Container

Deployment

Template for creating
pods in YAML file

Basic YAML – Deployment File

Pod

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
  - name: my-cont
    image: nginx:latest
```

To apply this YAML file to your Kubernetes cluster, you can use the `kubectl apply` command:

```
kubectl apply -f pod-definition.yaml
```

Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-deployment
spec:
  replicas: 2
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
      - name: my-cont
        image: nginx:latest
```

specify the Pod's metadata and containers.

to manage and select resources, such as in Services or ReplicaSets.

YAML Configuration File in Kubernetes

Three parts of Configuration File:

1. Metadata [Mentioned in config file]
2. Specification [Mentioned in config file]
3. Status [Not present in Config file]
 - Automatically generated and added by Kubernetes
 - Matches Desired state vs Actual State
 - [Ex: 2 replicas mentioned in config file (desired) vs 1 replica running (actual)] -> Self-Healing feature
 - 'etcd' component holds the current state of kubernetes cluster, which is displayed by Status

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-deployment
spec:
  replicas: 2
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
      - name: my-cont
        image: nginx:latest
```



Kubernetes Dashboard

View the created application in Minikube dashboard

Command: **minikube dashboard**

See 'Deployments', 'Pods', 'Replica Sets', 'services', and view their metadata

Kubernetes Demo

File: “Kubernetes Basics - MiniKube”



Deployment Strategies [Theory Only]

Deployment Strategies



Zero-Downtime Releases

Strategies that organisations use in production to ensure Zero-downtime deployments.

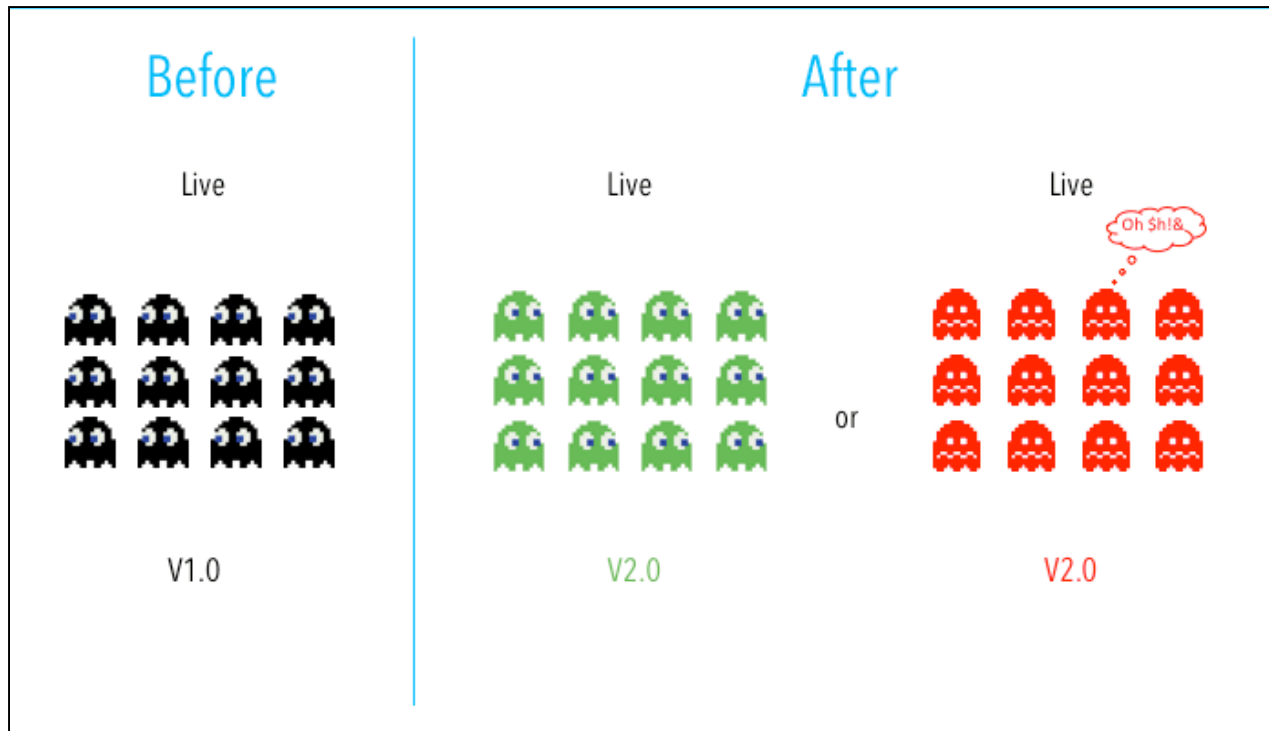
Also known as **hot deployment**, is one in which the actual process of switching users from one release to another happens nearly instantaneously.

1. Basic
2. Ramped
3. Blue/Green
4. Canary

1. Basic Deployment



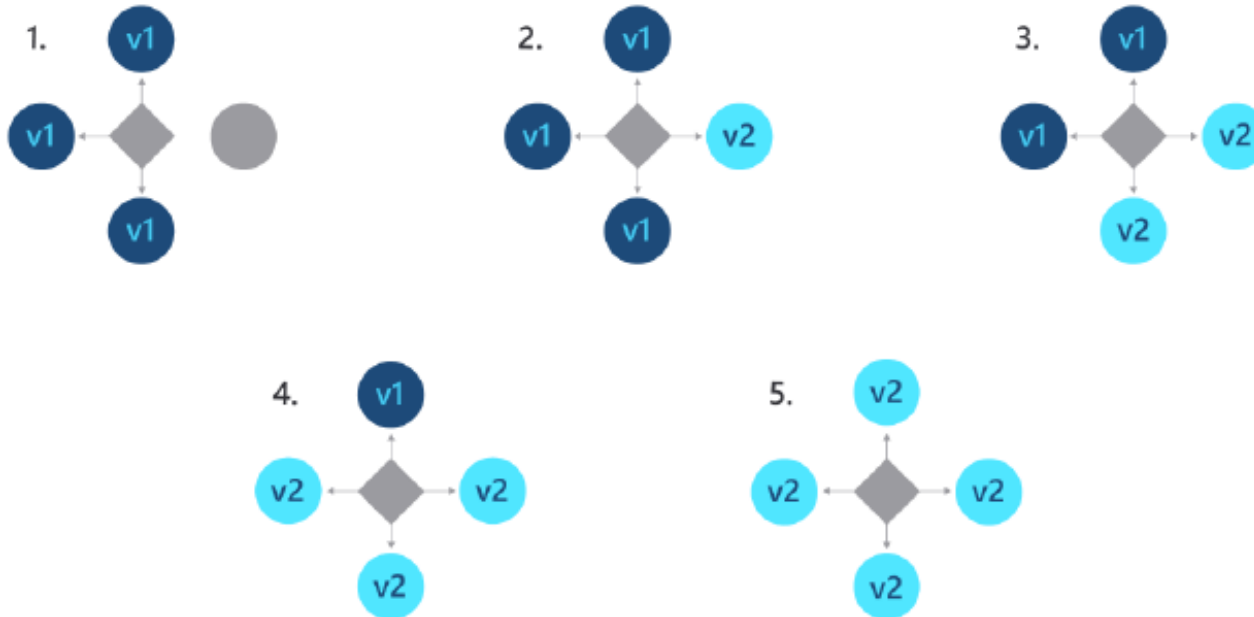
In a basic deployment, all nodes within a target environment are updated at the same time with a new service or artifact version. Because of this, basic deployments are not outage-proof and they slow down rollback processes or strategies. [Preferable for very small applications / services]



2. Ramped (also known as rolling-update or incremental)



- This deployment slowly replaces one node at a time to avoid downtime.
- Version 2 is slowly rolled out and replaces version 1
- **Default deployment strategy in Kubernetes**



2. Ramped (also known as rolling-update or incremental)



- Updates running instances of an application with the new release. All nodes in a target environment are incrementally updated with the service

Pros

- The benefits of a rolling deployment are that it is relatively simple to roll back, less risky than a basic deployment

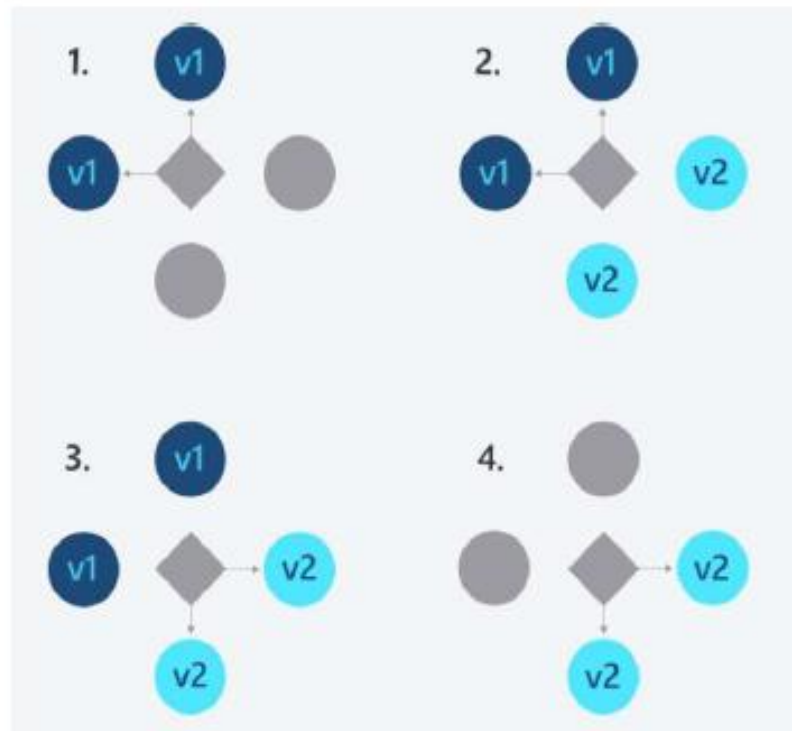
Cons

- Since nodes are updated in batches, rolling deployments require services to support both new and old versions of an artifact.
- Verification of an application deployment at every incremental change also makes this deployment slow.

3. Blue / Green



- In this you release a new version of your application or workflow while your current version is still running.
- Version 2 is released alongside version 1, then the traffic is switched to version 2.



3. Blue / Green – Why the name?

- Utilizes two identical environments, a “blue” (aka staging) and a “green” (aka production) environment with different versions of an application or service. Quality assurance and user acceptance testing are typically done within the blue environment that hosts new versions or changes. User traffic is shifted from the green environment to the blue environment once new changes have been tested and accepted within the blue environment.



3. Blue / Green



Pros

- It is simple, fast, well-understood, and easy to implement.
- Rollback is also straightforward, because you can simply flip traffic back to the old environment in case of any issues.
- Blue-green deployments are therefore not as risky compared to other deployment strategies

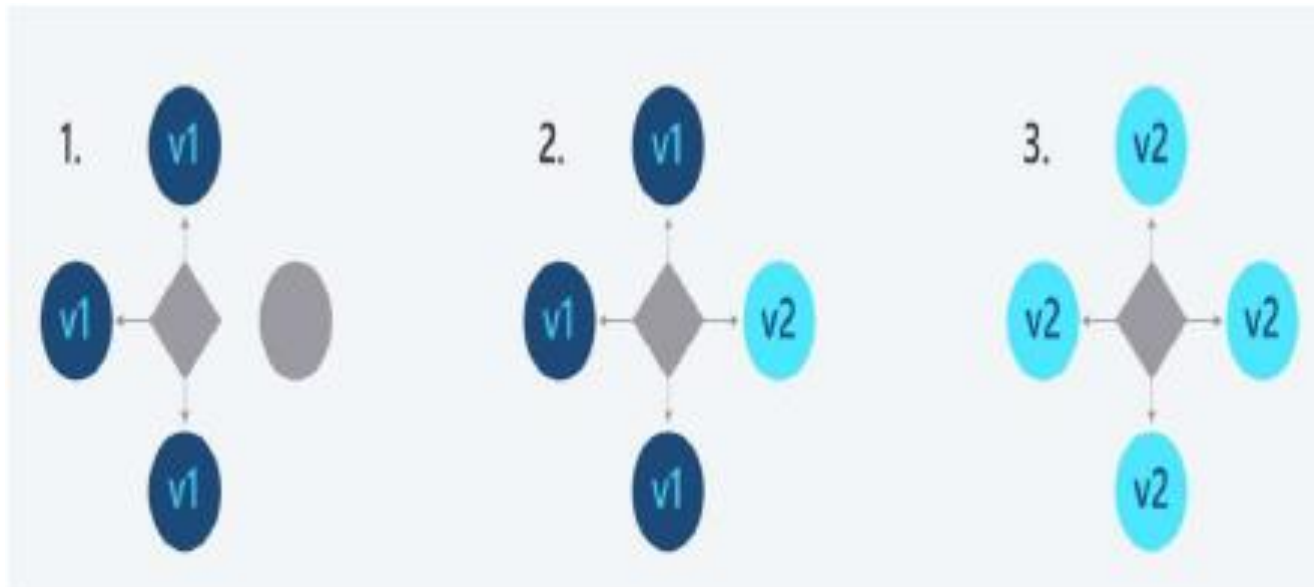
Cons

- Cost is a drawback to blue-green deployments. Replicating a production environment can be complex and expensive, especially when working with microservices.
- Quality assurance and user acceptance testing may not identify all of the anomalies or regressions either, and so shifting all user traffic at once can present risks

4. Canary



- Allow the customers to test your deployment by releasing the new version to a **small group of them**.
- Version 2 is released to a **subset of users**, then proceed to a full rollout.
- Ex: Facebook release of a new feature only to Asia Pacific region; if successful, then role out to other regions





BITS Pilani
Pilani Campus



Demo

Demo I



- How to dockerize and run a typical “Machine Learning” project?
- The ML project (code + ops) needs to be containerized
- The prediction should be available via an API endpoint, so that any end-user can make predictions on the ML model.

Demo



- **FastAPI:** Defines a prediction API using a POST endpoint that takes input data, preprocesses it, makes a prediction using a trained model, and logs metrics to MLflow. Runs at localhost:8000 port.
- **MLflow:** Configured as a tracking server accessible at <http://mlflow-service:5000> to log model parameters, metrics, and artifacts, allowing the FastAPI app to record and monitor model performance.
- **Dockerfile:** Sets up the environment by installing dependencies from requirements.txt, copies the application files, and configures Uvicorn to serve the FastAPI app on port 8000.
- **Docker Compose:** Orchestrates multi-container deployment with FastAPI and MLflow services, specifying network configurations and inter-service dependencies to enable seamless communication between FastAPI and MLflow.

Folder - “MLFlow_Covid_FastAPI-Endpoint_ImageDeployment”

docker-compose.yml file



- 2 services (fastapi, mlflow)
- fastapi needs to build from Dockerfile (build: .) and expose port 8000
- mlflow uses a prebuilt image and exposes port 5000
- fastapi depends_on mlflow (starts after mlflow)

Demo II



Dockerize a Fine Tuned Model available on Open AI

my_openai_ml_app/

- |-- app.py # FastAPI or Flask app for serving predictions
- |-- dockerfile # Docker configuration
- |-- requirements.txt # Dependencies
- |-- .env # Environment variables (optional)

Code -> “FineTunedModel-Dockerize”

Dockerizing a RAG Model



Code will have three parts:

1. Retriever Component: A vector store (e.g., FAISS, Elasticsearch) that indexes and retrieves relevant documents.
2. Generator Component: A language model (e.g., OpenAI GPT-4 or other LLMs) that processes the retrieved documents and generates a response.
3. API Wrapper: An application (e.g., FastAPI or Flask) that handles user inputs and orchestrates the RAG workflow.

```
rag_project/  
|-- retriever.py  
|-- generator.py  
|-- app.py          # FastAPI app  
|-- dockerfile  
|-- requirements.txt # Dependencies  
|-- .env            # Env variables
```



Thank You!